

Student: Sydoruk Tymofii K12521467

- ① $i = n - O(1)$
happens once and does not include any complexity time running.

while $i > 1$:

~~map~~ $i = i / 4$

loop which works until $i > 1$ and with
K times it divides into 4 so, $i = \frac{n}{4^K} \leq 1$
 $n \leq 4^K$

to find number of K perform: $K \geq \log_4(n)$

so: $\log_4(n) \cdot O(1) = O(\log_4(n))$

due to $\log_k(n) = \frac{\log(n)}{\log(k)}$ and ignoring constants:

Runtime: $O(\log n)$

- ② for i in range $(0, n, 2)$: Start: 0; End: n ;
Step: 2; so it's $\frac{n}{2} \Rightarrow O(n)$.

for j in range $(0, 3 \cdot n, 1)$: ~~8n~~ Number of iterations
is $3n$ meaning $O(n)$

inner loop for k in range $(n, 1, -1)$: Goes in the
other direction $n-1$ $\Rightarrow O(n)$.

If statements part is always $O(1)$ here.

Due to the nested loops we multiply them all,
 $(\frac{n}{2}) \cdot (3n) \cdot (n-1) = \frac{3n^3}{2} - \frac{3n^2}{2}$

Delete constants $= n^3 - n^2 = \max(n^3 - n^2) = n^3$,
 $O(n^3)$

③ a) for i in range $(1, n \cdot n + 1)$: from ~~1~~ to $n \cdot n + 1$ we perform n^2 steps meaning $O(n^2)$
for j in range $(i, n+1)$: it ~~it~~ would only work if $i \leq n$, because if $i > n$ the loop won't run.

$$\text{inner loop sum} = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2} \Rightarrow O(n^2)$$

$$\text{Inner} + \text{Extrol} = O(n^2) + O(n^2) = \underline{O(n^2)}.$$

④ b) for i in range (n) : from 0 to $n-1 \Rightarrow \underline{O(n)}$,
while $j \leq n$:

$j^* = 3$ in multiplication we have
Runtime $\log_3 n \Rightarrow \underline{O(\log n)}$.

while $k < n$

$k += 1$

In addition we have $O(n)$
because the amount of steps is $n-1$.
 $k=1$

Counting all steps:

$O(n) \cdot O(\log n) \cdot O(n)$ — They are all internal

$$\underline{O(n^2 \log n)}$$

c) for i in range (n, 0, -1): n-1 steps $\Rightarrow O(n)$

for j in range (1, n, 1): n steps $\Rightarrow O(n)$.

for i in range (0, n, 1): n steps $O(n)$

for j in range (m, m-1, -1): perform only once as starts with m and next is m-1.

$\Rightarrow O(1)$

for i in range (1, m, 10): $\frac{m}{10} \Rightarrow O(m)$.

Sum = $O(n) \cdot O(n) + O(n) + O(m) = O(n^2 + m)$

d) if $a > b$ and $b > c$:

for i in range (b, 0, -1): $\Rightarrow O(b)$

for j in range (b): $\Rightarrow O(b)$

if $a > c$:

for i in range (a): $\Rightarrow O(a)$ (ignore)

else:

for i in range (a * a): $\Rightarrow O(a^2)$

$O(a^2 + b^2)$

elif $a < b < c$:

for i in range (c, a): Here a is smaller than c so it won't perform.

$O(1)$.

else:

for i in range (a+5, a, -1): Steps inverse: 5

$O(1)$ Sum = $O(a^2 + b^2)$

(4.) a) We consider $K=1$ first
while $K < j$:

$K^* = 2$ Cycle stops at $K \geq j$

So it's $\log_2(j) \Rightarrow \underline{O(\log n)}$.

for j in range $(n, 0, -2)$: $\frac{n}{2} \Rightarrow \underline{O(n)}$.

$i = 1$

while $i < n$:

...

$i^* = 100$ Cycle ends at $100^n \geq n. \Rightarrow$

$\Rightarrow \log_{100}(n) \Rightarrow O(\log n)$.

Sum = $O(\log n) \cdot O(n) \cdot O(\log n) = \underline{O(n \log^2 n)}$